[5]

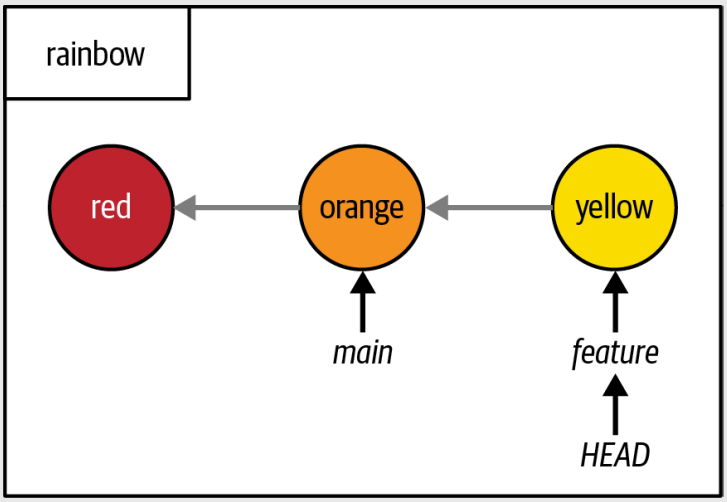
Merging

In the last chapter, you learned about branches, and we discussed how they allow you to work on the same project in different ways and to collaborate with other people on a project.
In this chapter, you are going to learn about integrating changes from one branch into another. In Git, there are two ways to do this: merging and re-basing. We will cover rebasing in Chapter 11; for now we will focus on merging. This chapter will introduce the two types of merges (fast-forward merges and three-way merges), and you will carry out a fast-forward merge.
In the process, you will also learn about how Git protects you from losing any uncommitted changes, how changing branches may change the contents of the working directory, and how to check out commits directly.

State of the Local Repository

At the start of this chapter, you should have three commits and two branches in your rainbow repository, and you should be on the feature branch. The current state of the rainbow repository is illustrated in Visualize it 5-1.

[VISUALIZE IT 5-1]



The rainbow repository at the start of Chapter 5, with three commits and two branches

Introducing Merging

In Chapter 4, you created your first branch, feature, and started working on that branch. Branches are a powerful feature of Git, and it's great that they allow us to work on different parts of a project independently. But how can you then combine the work you have done with the main branch once you're ready?

Merging in Git is one way you can integrate the changes made in one branch into another branch. In any merge, there is one branch that you are merging, called the *source branch*, and one branch that you're merging into, called the *target branch*. The source branch is the branch that contains the changes that will be integrated into the target branch. The target branch is the branch that receives the changes and is therefore the only one that is altered in this operation. Let's take a look at how to use merging in Example Book Project 5-1.

EXAMPLE BOOK PROJECT 5-1

Suppose that in my Book project, I decide together with my editor that every time I start to work on one of the chapters, I will make a secondary branch off the main branch. I also agree with my editor that I will merge the secondary branch into main only after they have reviewed my work on the branch.

For example, let's say that I decide to work on chapter 4. I can make a branch called chapter_four off the main branch, work on this branch, and then, when I think I've made significant headway, submit my work to my editor. Once they approve the changes, I can merge the chapter_four branch into the main branch in my local repository.

Example Book Project 5-1 illustrates that when you use branches, it's also essential to learn how to merge your work back together. Now, let's explore the different types of merges that exist.

Types of Merges

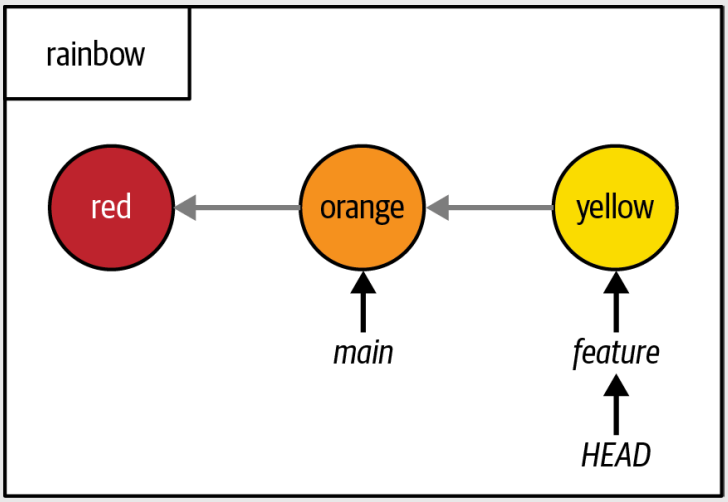
There are two types of merges:

- Fast-forward merges
- Three-way merges

The factor that determines which of these types of merges will take place when you merge the source branch into the target branch is whether the development histories of the two branches have diverged. A branch's development history can be traced by following the parent links of commits.

In Chapter 4, I explained that in the Repository Diagrams a gray arrow pointing from one commit to another indicates a parent link (that is, the arrow points backward from the child commit to the parent commit). In Visualize it 5-2, you can see that in the rainbow repository, the parent commit of the orange commit is the red commit. By the same logic, the orange commit is the parent commit of the yellow commit.

[VISUALIZE IT 5-2]



Parent links in the rainbow repository

The development history of a branch begins with the commit it points to, and extends backward through the chain of commits. In Visualize it 5-2, you can see that the development history of the main branch is therefore made up of the orange commit and the red commit, while the development history of the feature branch is made up of the yellow commit, the orange commit, and the red commit.

To illustrate the difference between fast-forward merges and three-way merges, let's start by going over an example of a fast-forward merge in Example Book Project 5-2.

From Example Book Project 5-2 we can see that a fast-forward merge is a type of merge that occurs when the development histories of the branches involved in the merge have not diverged—in other words, when it is possible to reach the target branch by following the parent links that make up the commit history of the source branch. During a fast-forward merge, Git takes the pointer of the target branch and moves it to the commit of the source branch.

EXAMPLE BOOK PROJECT 5-2

Suppose there are two commits on the main branch in my Book project repository, commits A and B, as seen in Figure 5-1.

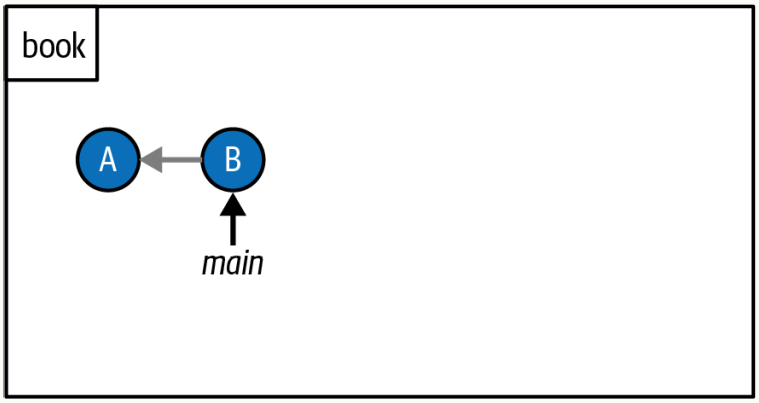


FIGURE 5-1

The commit history of the book repository with one branch, the main branch

Next, let's assume I create the chapter_six branch to work on chapter 6 of my book and I add commits C, D, and E to the chapter_six branch, as seen in Figure 5-2.

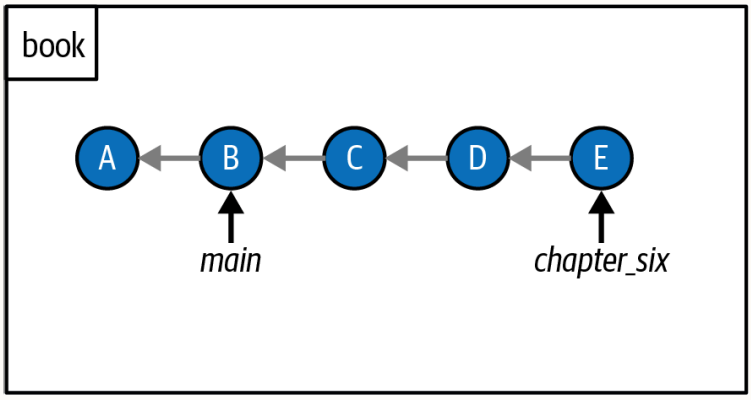


FIGURE 5-2

The commit history of the book repository after I work on the chapter_six branch

If I follow the parent links of the main branch backward, I can see that the branch is made up of commits A and B. In other words, the development history of the main branch consists of commits A and B. On the other hand, the development history of the chapter_six branch consists of commits A, B, C, D, and E.

If we can reach one branch through the commit history of another branch, we say that the development histories of the branches *have not diverged*. If I follow the parent links from the chapter_six branch, which points to commit E, backward, I reach the main

branch, which points to commit B. Therefore, the main branch and the chapter_six branch have not diverged.

If I were to now merge the chapter_six branch into the main branch, a *fast-forward merge* would occur. During the fast-forward merge, the main branch pointer would move forward to point to the commit that the chapter_six branch points to, which is commit E, as seen in Figure 5-3.

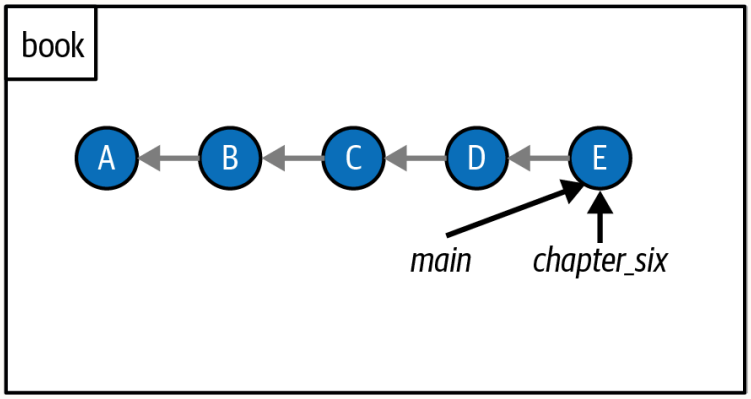


FIGURE 5-3

The commit history after I merge the chapter_six branch into the main branch in the book repository

In this merge example, chapter_six is the source branch and main is the target branch. Figure 5-3 shows that the main branch pointer simply moved forward from commit B to commit E. This is why these kinds of merges are called "fast-forward" merges.

Next, let's go over an example of a three-way merge in Example Book Project 5-3.

From Example Book Project 5-3 we can see that a three-way merge is a type of merge that occurs when the development histories of the branches involved in the merge have diverged. Development histories *have* diverged when it is *not* possible to reach the target branch by following the commit history of the source branch. In this case when you merge the source branch into the target branch, Git performs a three-way merge, creating a merge commit to tie the two development histories together; it then moves the pointer of the target branch to the merge commit.

EXAMPLE BOOK PROJECT 5-3

Suppose that the last two commits on the main branch in my book repository are commits F and G, as seen in Figure 5-4.

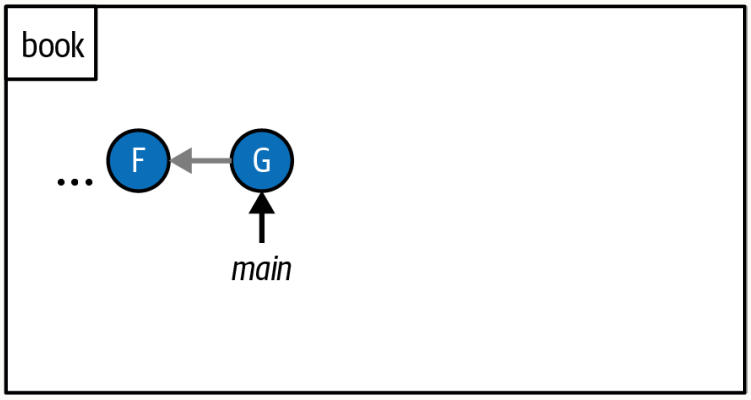


FIGURE 5-4

The commit history of the book repository with the last two commits on the main branch

Now suppose I decide to make a chapter_eight branch to work on chapter 8 of my book, and I make commits H, I, and J. At the same time, however, I add some work to the main branch, and it now points to commit L. This is illustrated in Figure 5-5.

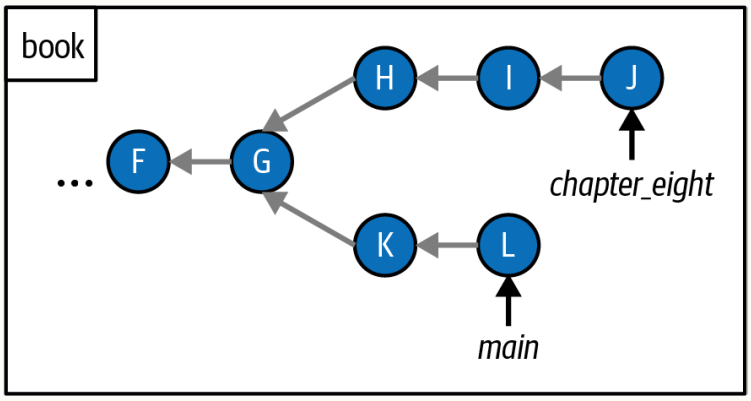


FIGURE 5-5

An example of a commit history where work has been added to the main branch and the chapter_eight branch

In Figure 5-5, you can see that the development history of the chapter_eight branch is made up of commits F, G, H, I, and J. On the other hand, the commit history of the main branch is made up of commits F, G, K, and L. There is no way to follow the parent links (represented by gray arrows) of the chapter_eight branch backward to reach the commit that the main branch points to, which is commit L. In Git, to describe this situation, we say that the development histories of the branches *have diverged*.

If I merge the chapter_eight branch into the main branch, it can't be a fast-forward merge because there is no way to just move the branch pointer forward to combine these two development histories. Instead, a *merge commit* (represented by commit M) will be created to tie the two development histories together, as shown in Figure 5-6. A merge commit is a commit that has more than one parent. This is an example of a *three-way merge*.

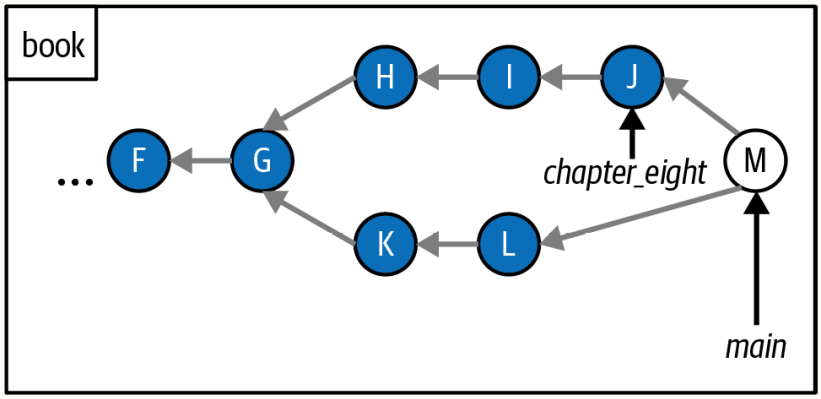


FIGURE 5-6

The commit history after I merge the chapter_eight branch into the main branch

In Figure 5-6, commit M points back to both commit J and commit L. The reason that this kind of merge is called a three-way merge is because in order to carry out the merge, Git will take a look at the two commits that the branches involved in the merge are pointing to—in the case of the book repository, commit J and commit L—as well as the commit that is the common ancestor of these two commits, which in this case is commit G. Hence the name, "three-way" merge.

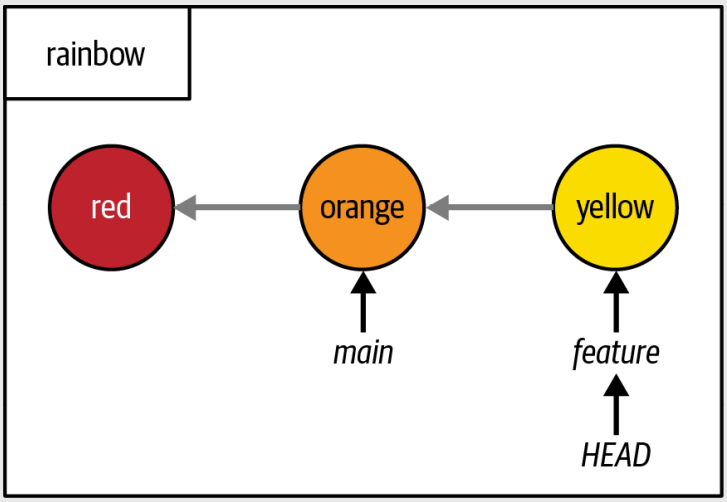
Three-way merges are a more complex type of merge where you may experience *merge conflicts*. These arise when you merge two branches where different changes have been made to the same parts of the same file(s), or if in one branch a file was deleted that was edited in the other branch.

Chapter 9 of this book is dedicated to explaining three-way merges in more depth, and in Chapter 10 you will learn more about merge conflicts. In each chapter, you will go through a hands-on example. In this chapter, you will practice carrying out a fast-forward merge in the Rainbow project.

Doing a Fast-Forward Merge

To practice merging in the rainbow repository, you will merge the feature branch into the main branch. The feature branch is the source branch and the main branch is the target branch. In Visualize it 5-3, you can see that you can reach the main branch by following the commit history of the feature branch; therefore, this will be a fast-forward merge.

[VISUALIZE IT 5-3]



In the rainbow repository, the development histories of the main branch and the feature branch have not diverged

There are two steps involved in doing a merge:

1. Switch onto the branch that you want to merge into (the target branch).
2. Use the git merge command and pass in the name of the branch you're merging (the source branch).

[SAVE THE COMMAND]

git merge <branch_name>

Integrate changes from one branch into another branch

This section will walk you through performing your first merge. In the process, you will learn two more important lessons about Git: first, that Git protects you from losing any work you have done in files that you have not committed, and second, that changing branches may cause the contents of the working directory to change.

SWITCHING ONTO THE BRANCH YOU ARE MERGING INTO

The first step of doing a merge is to switch onto the target branch. You are going to merge feature into main, so you need to switch onto the main branch. Recall the three things that happen when you use the git switch (or git checkout) command to switch onto a branch:

1. It changes the HEAD pointer to point to the branch you are switching onto.
2. It populates the staging area with all the files and directories that are part of the commit you are switching onto.
3. It copies the contents of the staging area into the working directory.

As these three steps indicate, if the branches point to different commits, changing branches also changes the contents of your working directory. You'll see this in practice momentarily, but first let's look at the other important feature I mentioned.

Git protects you from losing uncommitted changes

I just mentioned that switching branches changes the contents of your working directory. But what if you have modified files in your working directory (in other words, files that you have edited) that you have not yet committed? Will you lose all the work you've done in those files if you switch branches? Thankfully not! Git protects you from losing uncommitted changes.

If Git detects that switching branches will cause you to lose uncommitted changes in your working directory, then it will stop you from switching branches and present you with an error message. However, this happens only if the files that contain uncommitted changes have conflicting changes in the branch you are switching onto.

To illustrate why this is important, let's look at Example Book Project 5-4.

EXAMPLE BOOK PROJECT 5-4

Suppose I want to work on two different approaches for chapter 5 in my book repository. This means I'll be working on the chapter_five.txt file. I make two branches off the main branch, one called chapter_five_approach_a and one called chapter_five_approach_b.

First I go onto the chapter_five_approach_a branch and work on approach A. I make a couple commits. In these commits I'm editing the chapter_five.txt file.

Next, I decide to switch onto the chapter_five_approach_b branch to work on approach B in my text editor. While on the chapter_five_approach_b branch, I edit the chapter_five.txt file extensively, but I forget to actually make any commits on the branch (in other words, to properly save my work).

At some point, I decide I want to switch back to the chapter_five_approach_a branch to check something I had done on that branch. If Git simply allowed me to switch branches, then the version of the chapter_five.txt file in my working directory that I was working on in the chapter_five_approach_b branch would be replaced by the version of the file in the chapter_five_approach_a branch, and I would lose *all* my changes that I had worked on in approach B because I forgot to commit them.

Luckily, Git won't let this happen. Instead, it will warn me that I have uncommitted changes and it will remind me to make a commit so I won't lose them.

For a hands-on example in the Rainbow project of editing a file and witnessing how Git protects you from losing uncommitted changes, go on to Follow Along 5-1.

[FOLLOW ALONG 5-1]

1. rainbow $ **git status**
   On branch feature
   nothing to commit, working tree clean

2. In the rainbow project directory in your text editor, in the rainbowcolors.txt file, add “Green is the fourth color of the rainbow.” on line 4. Then save the file.

3. rainbow $ **git status**
   On branch feature
   Changes not staged for commit:
   (use "git add <file>..." to update what will be committed)
   (use "git restore <file>..." to discard changes in working directory)
   modified: rainbowcolors.txt
   no changes added to commit (use "git add" and/or "git commit -a")

4. rainbow $ **git switch main**
   error: Your local changes to the following files would be overwritten by checkout:
   rainbowcolors.txt
   Please commit your changes or stash them before you switch branches.
   Aborting

What to notice:

- In step 1, the git status output shows you that there are no modified files in the working directory.
- In step 3, the git status output indicates that the rainbowcolors.txt file is a modified file in the working directory.
- In step 4, you witness how Git *does not* allow you to switch branches. It warns you that Your local changes to the following files would be overwritten by checkout and it tells you to Please commit your changes or stash them before you switch branches.

If Git had allowed you to switch branches, then the version of the rainbowcolors.txt file in the working directory that mentions the colors red, orange, yellow, and green would

have been replaced by the version of the rainbowcolors.txt file in the orange commit that the main branch points to, which only mentions the colors red and orange. This means that you would have lost *all* the work you did in adding the color green to your list of colors.

Instead, Git protects you from losing your work in the rainbowcolors.txt file by warning you to commit your changes.

[NOTE]

Git will not prevent you from switching branches if you make changes to files without saving them in the text editor, because those files will be considered unmodified files. So, always remember to save files in your text editor when you're done working on them!

Now, you'll continue with carrying out the first merge in the Rainbow project. In this case, let's assume you aren't yet ready to add notes about the color green. Go ahead to Follow Along 5-2 and remove the sentence about that color, so that there are no modified files in your working directory.

[FOLLOW ALONG 5-2]

1. In the rainbow project directory in your text editor, remove the line "Green is the fourth color of the rainbow." from the rainbowcolors.txt file and save the file. Make sure to remove any extra lines or spaces that you might have added.

2. rainbow $ **git status**
   On branch feature
   nothing to commit, working tree clean

What to notice:

- The git status command indicates that your working directory no longer has any modified files.

Next, you're going to carry out the first step of a merge, which is to switch onto the branch you are merging into. While you do this, you will witness how switching branches changes the files in your working directory.

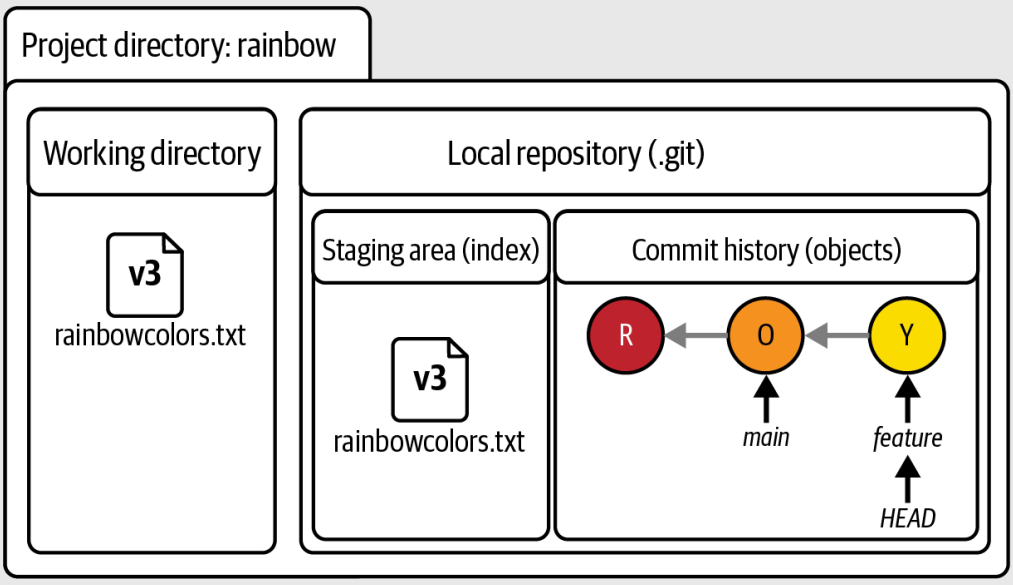
Switching branches changes files in the working directory

To see how switching branches changes the files in the working directory, take a look at Visualize it 5-4, which uses the Git Diagram introduced in Chapter 2 to show what the different areas of the rainbow project directory look like at the moment.

[NOTE]

In Visualize It diagrams where there isn't enough space to include the full name of a commit, I'll use an abbreviated form of the name instead: for example, R for red, O for orange, and so on. For a full list of the abbreviations used for each commit, see Table P-1 in the Preface.

[VISUALIZE IT 5-4]



The rainbow project directory before you switch from the feature branch onto the main branch

What to notice:

- The version of the rainbowcolors.txt file in the working directory and staging area is the one that mentions the colors red, orange, and yellow. It is represented as version 3 (v3) of the file.

Now, go to Follow Along 5-3 to switch from the feature branch onto the main branch in the rainbow repository (which points to a different commit) and observe how the contents of your working directory change.

[FOLLOW ALONG 5-3]

1. Make sure you have the rainbowcolors.txt file open in your text editor window, and place it next to your command line window so that you have a view of both of them when you execute the upcoming commands. Look at the contents of the rainbowcolors.txt file.

2. rainbow $ **git switch main**
   Switched to branch 'main'

3. Look at the contents of the rainbowcolors.txt file.

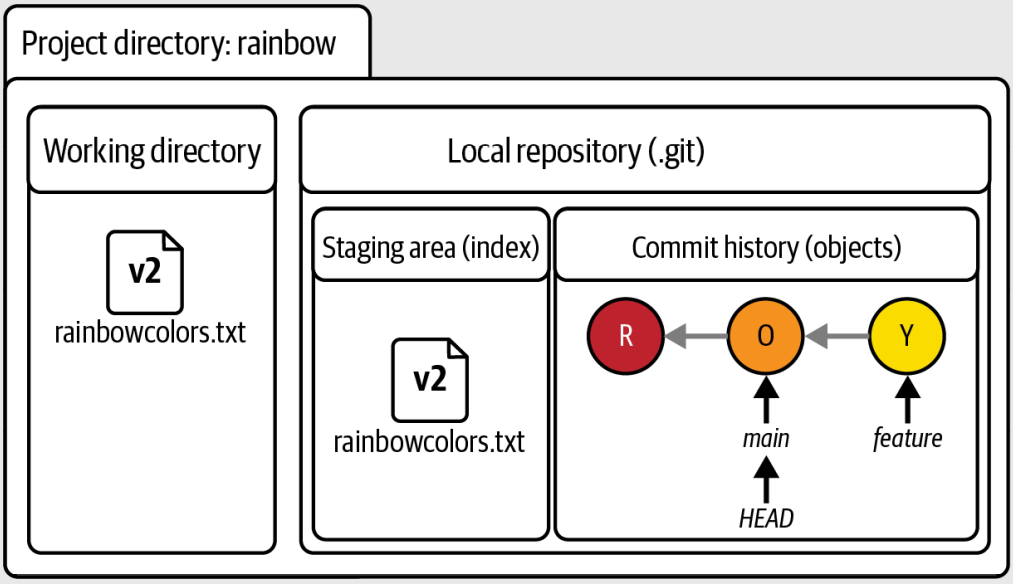
4. rainbow $ **git log**
   commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD -> main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

What to notice:

- The version of the rainbowcolors.txt file that was in your text editor before you switched onto the main branch mentioned the colors red, orange, and yellow. The version of the rainbowcolors.txt file that is in your text editor after you switch onto the main branch mentions only the colors red and orange. This is illustrated in Visualize It 5-5.

- In step 4, the git log output shows only the red and orange commits. It no longer shows the yellow commit.

[VISUALIZE IT 5-5]



The rainbow project directory after you switch from the feature branch onto the main branch

What to notice:

- You are on the main branch and it points to the orange commit, which includes the version of the rainbowcolors.txt file that mentions the colors red and orange. This is represented as version 2 (v2).
- v3 of the rainbowcolors.txt file has been replaced by v2 of the file in both the staging area and the working directory.

You have just explicitly observed that switching branches changes the contents of the working directory. Before you go on to the second step of doing the merge, let's briefly touch upon why the git log output in step 4 of Follow Along 5-3 shows only the red and orange commits.

Viewing a list of all commits

In “Viewing a List of Commits” on page 40, I mentioned that the git log command shows a list of commits in reverse chronological order. However, in reality, it shows only

a list of commits that are reachable by following the parent links from the commit you are on when you execute the command. To see a list of commits for *all* the branches in your local repository, you must use the git log command with the --all option.

[SAVE THE COMMAND]

git log --all

> Show a list of commits in reverse chronological order for all branches in a local repository

Go to Follow Along 5-4 to see a list of the commits for all the branches in your local repository.

[FOLLOW ALONG 5-4]

1 rainbow $ **git log --all**

```
commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (feature)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 10:09:59 2022 +0100
yellow
commit 7acb333f08e12020efb5c6b563b285040c9dba93 (HEAD -> main)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:42:07 2022 +0100
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red
```

What to notice:

- The git log --all output shows the red, orange, and yellow commits.

Now that you are on the main branch, you can move on to the second step of doing a merge.

USING THE GIT MERGE COMMAND TO EXECUTE A MERGE

Go to Follow Along 5-5 to execute the merge by passing the name of the branch you are merging—the source branch, which is feature—to the git merge command.

[FOLLOW ALONG 5-5]

1. Make sure your text editor window with a view of the rainbowcolors.txt file is open next to your command line window so that you can see both of them when you execute the upcoming commands. Look at the contents of the rainbowcolors.txt file.

2. rainbow $ **git merge feature**
   Updating 7acb333..fc8139c
   Fast-forward
   rainbowcolors.txt | 3 ++-
   1 file changed, 2 insertions(+), 1 deletion(-)

3. Look at the contents of the rainbowcolors.txt file.

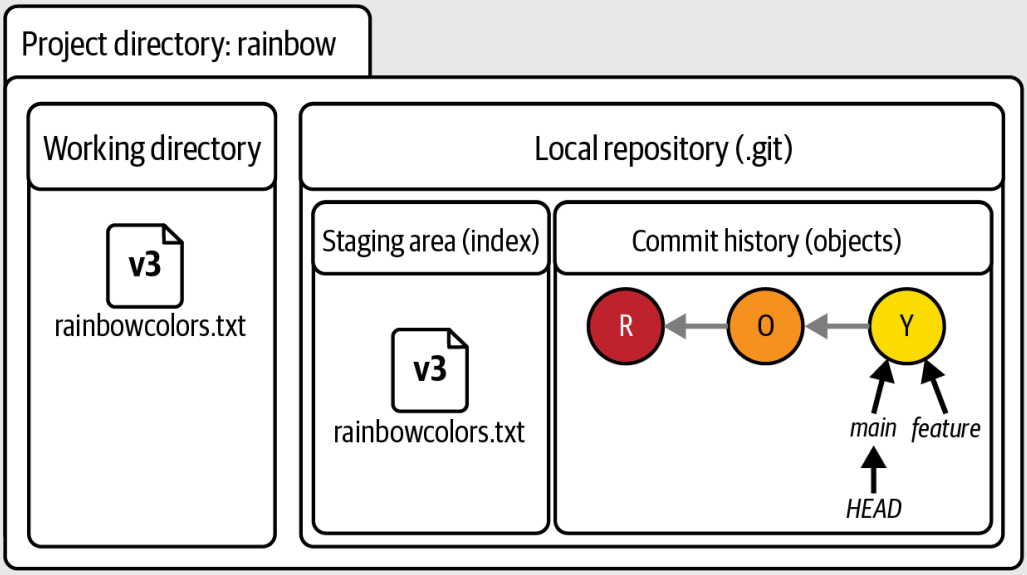
4. rainbow $ **git log**
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> main, feature**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow
   commit 7acb333f08e12020efb5c6b563b285040c9dba93
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

What to notice:

- The git merge output mentions Updating 7acb333..fc8139c and Fast-forward. This tells you that Git updated the commit the main branch points to, and it tells you that this was a fast-forward merge. Note that the commit hashes in your output will be different from the ones in this book.
- The git log output shows that main points to the yellow commit. In your text editor, you can see that your working directory contains the version of the rainbowcolors.txt file that is part of the yellow commit.
- You merged the feature branch into the main branch, but it still exists; it did not get automatically deleted.

All of these observations are illustrated in Visualize it 5-6.

[VISUALIZE IT 5-6]



The working directory and the local repository after merging feature into main

You merged feature into main, but in Git, merging a branch does not delete the branch. You must explicitly delete a branch if you no longer want to use it. For now, you will keep the feature branch, and we will go over how to delete branches in Chapter 8. The next topic I want to cover here is how to check out commits.

Checking Out Commits

In Chapter 4, I mentioned that the git checkout command may be used to switch branches as well as to carry out other actions. One of the other things you can do with the git checkout command is check out commits.

At the moment, you are on the main branch, which points to the yellow commit. But what if you want to look at an older version of your project? For example, what if you want to see the state of your project at the orange commit?

There is currently no branch pointing to the orange commit, so you can't switch onto it by switching onto a branch. Instead, you can choose to *check out* that commit by using the git checkout command and passing in the commit hash of the orange commit.

[SAVE THE COMMAND]

git checkout <commit_hash>

Check out a commit

When you do this, the git checkout command will carry out three actions that are similar to the ones described in Chapter 4 and earlier in this chapter:

1. It changes the HEAD pointer to point to the *commit* you are switching onto.
2. It populates the staging area with all the files and directories that are part of the commit you are switching onto.
3. It copies the contents of the staging area into the working directory.

The main difference between these steps and the steps mentioned previously is that in step 1 the HEAD pointer will point directly to a commit instead of pointing to a branch. This means that you will be in something that Git (scarily) calls *detached* HEAD *state*. This allows you to look at any commit—or, in other words, any version of your project—in your entire repository.

As these steps indicate, checking out commits changes the contents of the working directory in the same way that switching branches does.

[NOTE]

It is not recommended to make any changes to a repository while in detached HEAD state (that is, while not on a branch). You will usually want to make commits on branches because branches are easier to remember and refer to than commit hashes, and because Git was designed to be used with branches. Therefore, if you check out a commit, it is common to make and switch onto a new branch that will point to that commit, so that you're no longer in detached HEAD state.

Go to Follow Along 5-6 to check out the orange commit and observe what it's like to be in detached HEAD state.

[FOLLOW ALONG 5-6]

1. Retrieve the commit hash for your orange commit (you can copy this from your git log output). Make sure to use your commit hash in step 2 of this Follow Along.

2. rainbow $ **git checkout 7acb333f08e12020efb5c6b563b285040c9dba93**
   Note: switching to '7acb333f08e12020efb5c6b563b285040c9dba93'.
   You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.
   If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example:
   git switch -c <new-branch-name>
   Or undo this operation with:
   git switch -
   Turn off this advice by setting config variable advice.detachedHead to false
   HEAD is now at 7acb333 orange

3. rainbow $ **git log --all**
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**main**, **feature**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow
   commit 7acb333f08e12020efb5c6b563b285040c9dba93 (**HEAD**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

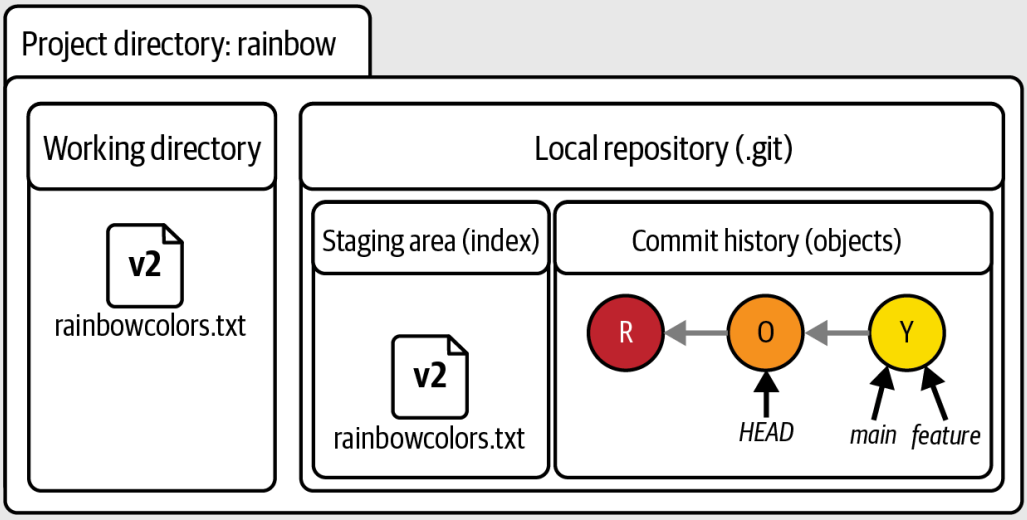
4. Look at the contents of the rainbowcolors.txt file.

What to notice:

- In step 2, the git checkout output tells you that You are in 'detached HEAD' state. The output also refers to a modification of the git switch command, which we will touch upon in the following section.
- In step 3, the git log output shows you that HEAD now points to the orange commit.
- The version of the rainbowcolors.txt file in your working directory is the version that is part of the orange commit, which mentions only the colors red and orange (represented by v2).

These observations are illustrated in Visualize it 5-7.

[VISUALIZE IT 5-7]



The working directory and the local repository after you check out the orange commit and enter detached HEAD state

[NOTE]

It may seem scary that the contents of the working directory change every time you switch branches or check out a commit directly—but remember that you have not lost anything. All of your commits are safely stored in the commit history, and you can always choose to switch to another branch or to check out another commit to see what you were looking at before.

You just observed what it is like to check out a commit directly instead of checking out a branch. Next, go to Follow Along 5-7 to switch back onto the main branch and exit detached HEAD state.

[FOLLOW ALONG 5-7]

1. rainbow $ **git switch main**
   Previous HEAD position was 7acb333 orange
   Switched to branch 'main'

2. rainbow $ **git log**
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> main**, **feature**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow
   commit 7acb333f08e12020efb5c6b563b285040c9dba93
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange
   commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:23:18 2022 +0100
   red

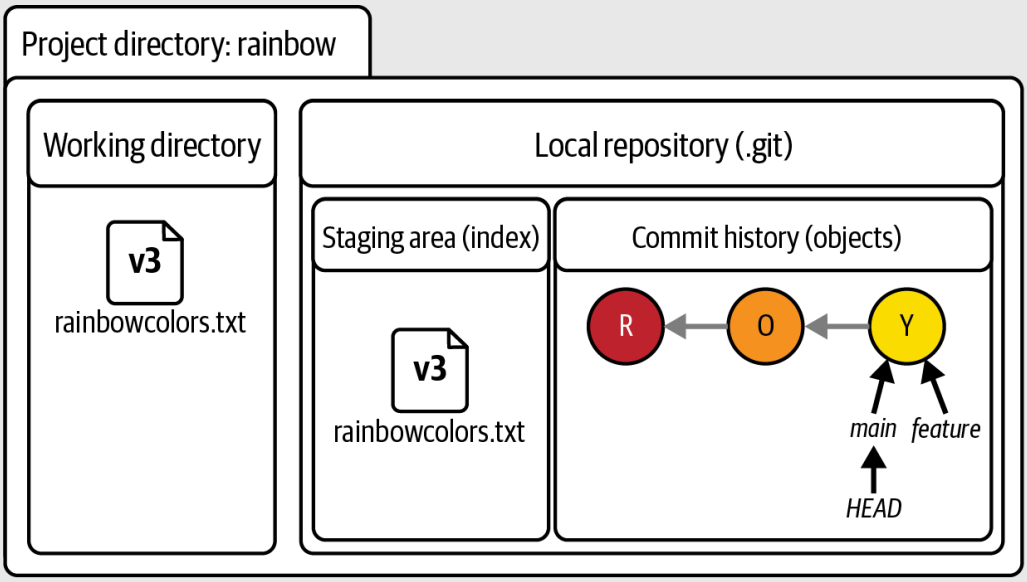
3. Look at the contents of the rainbowcolors.txt file.

What to notice:

- The version of the rainbowcolors.txt file in your working directory is the version that is part of the yellow commit, which mentions the colors red, orange, and yellow (represented by v3).
- You are back on the main branch.

This is illustrated in Visualize it 5-8.

[VISUALIZE IT 5-8]



The working directory and the local repository after you switch onto the main branch and exit detached HEAD state

Creating a Branch and Switching onto It in One Go

In Chapter 4, you learned to make a new branch by using the git branch command and to use the git switch (or git checkout) command to change onto that branch.

In step 2 of Follow Along 5-6, the git checkout output stated: If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example: git switch -c <new-branch-name>.

This is because it is actually possible to use the git switch or git checkout command to create a branch and switch onto it in one go. If you use the git switch command you must use the -c option (which stands for "create"), and if you use the git checkout command you must use the -b option.

[SAVE THE COMMAND]

git switch -c <new_branch_name>

Create a new branch and switch onto it

git checkout -b <new_branch_name>

Create a new branch and switch onto it

You will practice using this command in “Preparing to Make a Pull Request” on page 240, when you make a new branch.

Summary

In this chapter, we covered what merging is and why we do it. I introduced the two types of merges—fast-forward merges and three-way merges—and explained how the type of merge that will be carried out depends on the development histories of the branches involved in the merge.

Finally, you performed a fast-forward merge yourself in the rainbow repository, learning in the process that Git protects you from losing uncommitted changes, that changing branches may change the contents of the working directory, and how to check out commits.

This chapter marks the end of the first part of this book, in which you worked exclusively with local repositories. Next, in Chapter 6, you are going to prepare your hosting service account and authentication details in order to start working with remote repositories.

Previous chapter
Chapter 4. Branches
Next chapter
Chapter 6. Hosting Services and Authentication